

Specification Composition for LLM-Driven Python Verification: Why We Build on Iris

A short workshop paper for the Iris community

Gavin Mendel-Gleason
Scidonia

Abstract

Axiomander is a verification pipeline that turns ordinary, decorator-free Python into machine-checked proof obligations and discharges them through a ladder of mechanical tactics, SMT, and an LLM proof oracle. Its purpose is *vericoding*: a human iterates on a strong specification, an LLM derives the implementation (and the helper specifications needed to prove it), and the verifier delivers a deterministic verdict with a structured residual on failure. The viability of this loop rests entirely on *specification composition* – the ability to verify a caller against a callee’s *contract* rather than its body, to reuse proofs across edits, and to stub unverified libraries by their declared behaviour.

This paper argues that **separation logic, as realised by Iris, is the right foundation for that composition**, and reports our experience replacing an ad-hoc framing mechanism with a native Iris one. Our first backend lowered Python to a small imperative language (IMP) over a flat store and recovered locality through generated per-callee, per-variable *frame lemmas* over an explicit clobber operator. That mechanism works but scales badly: it produces WP-term blowup and brittle compiled-Ltac pattern matching, and it does not extend to aliasing, ownership transfer, or concurrency. We describe a second backend, **SnakeletLang/SnakeletWp**, an Iris language and weakest-precondition calculus (currently 0 Admitted) in which a callee’s footprint is a separating resource, framing is the separating conjunction, and a *contract table* lets calls be treated opaquely (as stubs) or transparently (unfolded). We give Snakelet’s full term language and weakest-precondition rule set, the contract-to-iProp compilation, the opaque-call rule that underpins stubbing, and a staged, syntax-directed proof generator whose stages fail independently and feed residual goals to SMT and an LLM. We close with the open questions we would most like the Iris community’s input on.

Keywords

separation logic, Iris, weakest preconditions, specification composition, stubbing, frame rule, LLM-assisted proof, Python

1 The setting: vericoding needs composition

Axiomander’s development model is a loop. The human writes a *strong specification* – a precondition/postcondition pair – as the primary artifact. An LLM proposes an implementation. The verifier reduces “does the code meet the spec?” to a proof obligation and returns either a certificate or a *structured residual* (the remaining goal with its hypotheses, plus a counterexample where one exists). The LLM repairs against the residual, not against the original natural-language prompt. Crucially, the LLM has freedom in *sub-specification*: to prove an entry point it may introduce helper

functions, each with its own contract, which are checked the same way.

For this loop to be usable on real codebases, three forms of composition must hold simultaneously:

- (1) **Modular verification.** A caller must be provable from a callee’s *contract* alone. If proving f required unfolding the bodies of everything f transitively calls, neither proofs nor LLM prompts would scale.
- (2) **Proof reuse under iteration.** Editing a function body must not invalidate its callers’ proofs as long as the function’s *contract* is unchanged. The verifier should behave like an incremental build system: *body changes invalidate local proofs; contract changes invalidate callers*.
- (3) **Stubbing.** Libraries we cannot or will not verify must be replaceable by their declared behaviour. A caller proves against the stub’s contract, which becomes an assumption; the trust boundary is the stub, written and reviewed once.

All three are, at bottom, the **frame property**: reasoning about one component must be insensitive to the parts of the state it does not touch. This is exactly what separation logic makes primitive, and it is why – after building framing the hard way once – we are migrating Axiomander’s proof backend to Iris.

2 First backend: framing by hand over a flat store

Our original backend lowers a verified subset of Python to a small imperative language IMP, with a state that is a flat finite map from variable names to values ($VZ \mid VBool \mid VUnit$, plus structural $VList \mid VTuple \mid VDict$, with mutation modelled on a parallel heap representation). The WP calculus is standard and proven sound in Rocq:

$$\begin{aligned} \text{wp}(\text{skip}, Q) &= Q, \\ \text{wp}(x := e, Q) &= Q[x \mapsto e], \\ \text{wp}(c_1; c_2, Q) &= \text{wp}(c_1, \text{wp}(c_2, Q)), \\ \text{wp}(\text{if } e \text{ } c_1 \text{ } c_2, Q) &= (e \Rightarrow \text{wp}(c_1, Q)) \\ &\quad \wedge (\neg e \Rightarrow \text{wp}(c_2, Q)). \end{aligned}$$

Calls are the interesting case. A call site $t := g(a)$ is verified against g ’s contract using a CCall rule. To recover locality – to let a caller conclude that variables it cares about survive the call – we model the callee’s effect with an explicit clobber operator that havoc the callee’s declared write set, and we must discharge, for every variable v , an obligation of the form

$$\begin{aligned} \forall v. v \notin (\text{target} :: \text{writes}) \Rightarrow \\ \text{lget } s \ v = \text{lget } (\text{clobber } (\text{lupd } s \ \text{target } r) \ \text{writes}) \ v. \end{aligned}$$

This is the frame rule, but encoded extensionally over a flat store. Two problems follow, both of which we hit in practice:

- **WP-term blowup.** Each successive call nests the full WP expansion inside the previous call’s postcondition. A function with a handful of sequential calls produces terms too large for the kernel to handle comfortably.
- **Brittle automation.** The single universally-quantified frame subgoal does not match cleanly in compiled Ltac, partly because coercions (e.g. an `ls` wrapper) normalise away in the `.vo` and defeat the source-level pattern.

Our mitigation was to generate, at the Python IR level, **one frame lemma per (callee, preserved-variable) pair**:

```
(* g writes ["result"]; the caller's "a" is
   preserved across t := g(a) *)
Lemma g_frame_a : forall (s : state) (r : Z),
  ~ In "a" ("t" :: "result" :: nil) ->
  lget s "a" =
    lget (clobber (lupd s "t" (VZ r))
          ("result" :: nil)) "a".
Proof. apply wp_ccall_frame. Qed.
```

The caller’s proof then replaces the monolithic $\forall v$ subgoal with a sequence of `apply g_frame_a. apply g_frame_b. ...`, each trivial and each failing independently with its own residual. This restores tractability and keeps the per-obligation, fail-locally discipline we want. But it is, frankly, a workaround: we are *reconstructing* the frame rule, one variable at a time, in a logic that does not have it. It does not model aliasing, it does not express ownership transfer, and it has no path to concurrency. The lesson is the familiar one: **if framing is load-bearing, the assertion logic should be substructural**.

3 Second backend: SnakeletLang and SnakeletWp over Iris

We have built a second backend in which framing is not reconstructed but inherited. **Snakelet** is the Iris *intermediate language* into which Axiomander lowers Python, and **SnakeletWp** is its weakest-precondition calculus, defined inside the Iris program logic. The state is an Iris heap; a mutable Python object field is a points-to assertion $l \mapsto v$, and ownership is a separating resource rather than an entry in a flat map; assertions are `iProp`. The next three subsections give Snakelet’s term language (Fig. 1), its operational semantics (Fig. 2), and its weakest-precondition rules (Fig. 3); the calculus currently carries **0 Admitted** across the entire rule set – values, binary operators, `let`, `if`, `load/store/alloc`, the two call forms, and the exception and loop rules whose postcondition ranges over a `Result`.

3.1 Snakelet: an intermediate language for Python in the large

Axiomander targets Python *in the large*, not a hand-picked subset. Ordinary code – assignments and expressions, conditionals, `while` and `for` loops, `try/except`, function and method calls, and the built-in container types – lowers directly to Snakelet, where every proof obligation is discharged. Rather than admit a small whitelist, the front end accepts Python broadly and *gates* only the genuinely pathological constructs – runtime metaprogramming

$$\begin{aligned}
 v &::= n \mid b \mid fl \mid s \mid l \mid () \mid \text{exn } s \mid v \\
 &\quad \mid [\bar{v}] \mid (\bar{v}) \mid \{\bar{v}; \bar{v}\} \mid \{\bar{v}\} \\
 op &::= + \mid - \mid \times \mid / \mid \% \mid = \mid \neq \mid < \mid \leq \mid > \mid \geq \\
 &\quad \mid \text{and} \mid \text{or} \mid \text{in} \mid \text{len} \mid \text{union} \mid \text{inter} \\
 e &::= v \mid x \mid \text{let } x = e \text{ in } e \mid op(e, e) \mid g(\bar{e}) \\
 &\quad \mid !e \mid e := e \mid \text{ref } e \mid \text{if } e \text{ then } e \text{ else } e \\
 &\quad \mid \text{raise } e \mid \text{try } e \text{ catch } x \Rightarrow e \\
 &\quad \mid \text{while } e \text{ e} \mid \text{for } x \text{ in } e \text{ do } e \\
 K &::= \text{let } x = \square \text{ in } e \mid op(\square, v) \mid op(e, \square) \mid !\square \mid \text{ref } \square \\
 &\quad \mid \square := v \mid e := \square \mid \text{if } \square \text{ then } e \text{ else } e \mid \text{raise } \square \\
 &\quad \mid \text{try } \square \text{ catch } x \Rightarrow e \mid \text{for } x \text{ in } \square \text{ do } e \\
 r &::= \text{val } v \mid \text{exn } s \mid v
 \end{aligned}$$

Figure 1: Snakelet term language: values v (integer, boolean, float, string, location, unit, exception object, and the structural list/tuple/dict/set), operators op , expressions e , single-item evaluation contexts K , and results r . Every K is *neutral* except `try  $\square$  ...`; $op(e, e)$ evaluates its right operand first. The core development additionally provides `FAA`, `fork`, and `dictionary get/set`.

(eval/exec, dynamic attribute mutation), reflection, and the like – rejecting them with a diagnostic instead of silently mistranslating. The Python-to-Snakelet lowering is the trusted boundary of the pipeline (intended to be extracted from Coq later); Snakelet itself is small enough to carry a complete operational semantics and WP calculus, which is what this section gives.

Snakelet is an expression language in administrative-normal form: every operator and call takes values or variables, and evaluation order is made explicit by `let`-binding and a per-item evaluation-context grammar. A store is a finite map from locations to values ($\sigma : \text{loc} \rightarrow v$); a mutable Python object field is a heap cell, exposed in the logic as $l \mapsto v$. The immutable Python containers – list, tuple, dict, set – are *structural* values, not heap objects, mirroring Dafny’s `seq` and `F*`’s `list`.

Two coordinated developments. The core language carries float literals, dictionary `get/set`, and the concurrency primitives `FAA/fork`; the exception-aware extension presented here adds first-class exceptions and carries the complete, machine-checked rule set. We present the latter because exceptions change the *shape of the judgement*. Following van Collem/de Vilhena/Krebbes (PLDI 2026), `raise v` on a value is a **stuck terminal** expression – not itself a value – so an uncaught `raise` sits in evaluation position as the program’s exceptional result. A program thus terminates in one of two ways, captured by the result type $r ::= \text{val } v \mid \text{exn } s \mid v$, and (Section 3.3) the WP postcondition ranges over r rather than over values. Figure 1 gives the term language.

3.2 Operational semantics

Reduction is small-step and factors into *pure* steps $e \rightsquigarrow e'$ (no heap, deterministic) and *heap* steps $(e, \sigma) \rightarrow (e', \sigma'; ef)$ (heap and calls), each lifted under an evaluation context by the two decomposition rules `CTX-P`/`CTX-H` (Figure 2). Contexts are single-item because the WP binds one frame at a time.

All context items are **neutral** except `try  $\square$  catch  $x \Rightarrow h$` . The unwinding rule `UNWIND`, $K[\text{raise } v] \rightsquigarrow \text{raise } v$, fires only for neutral K , so a `raise` propagates through `let`, operators, loads, and

the rest, but is *caught* by an enclosing try frame (TRYC) – exactly Python/ML exception propagation. Calls dispatch through a *contract table* entries: a name maps to either FunSpec $P\ Q$ (*opaque* – the call steps to some result satisfying Q only when P holds, so calling outside the precondition is stuck) or FunDef $\tilde{x}\ b$ (*transparent* – substitute the arguments into the body). The two modes share one table with one entry per name, so they are mutually exclusive by construction.

3.3 The weakest-precondition rules

SnakeletWp is a hand-rolled weakest precondition whose postcondition is $\Phi : \text{Result} \rightarrow \text{iProp}$. It is the guarded fixpoint of: if e is terminal ($\text{result}(e) = \text{Some } r$) then Φr holds under a fancy update; otherwise e is reducible in the current heap and, after one step, the state interpretation (the Iris gen_heap authoritative view) and the WP of the reduct are re-established. Ranging Φ over Result is the one structural change exceptions force; everything else is standard Iris. Figure 3 lists the rules, all proved with **0 Admitted**.

Three points. First, **composition goes through bindp**: the bind rule BIND threads a Result-typed continuation $\text{bindp } K\ \Phi$ defined by $\text{bindp } K\ \Phi\ (\text{val } v) = \text{WPE } K[v]\ \{\Phi\}$ and $\text{bindp } K\ \Phi\ (\text{exn } s\ v) = \Phi(\text{exn } s\ v)$ – on a value it keeps evaluating the context, on an exception it short-circuits to the exceptional postcondition. This is the convergence-critical lemma: it is what makes the exception WP *compose*, and with UNWIND/LETRAISE it delivers full propagation. Second, the **heap rules are orthogonal to exceptions**: LOAD/STORE/ALLOC are the usual Iris points-to rules, and the exceptional arm of Φ describes the heap *as it stood at the raise* – a function that mutates a cell then raises satisfies a postcondition whose $\text{exn } s$ arm asserts $l \mapsto v$ with the *mutated* value. Third, the **two call rules are the composition story** expanded in Section 3.6: CALL_OPAQUE is stubbing and modular verification (the caller establishes $P\ \vec{v}$, receives $Q\ \vec{v}\ v$, and never sees the body); CALL_DEF unfolds a first-party body.

For loops, WHILE/FORNIL/FORCONS unfold one iteration. On top of FORCONS we derive a structural fold rule that carries a user invariant P over the *remaining suffix* of the iterated list and re-establishes it (or escapes through the exceptional arm) at each element:

$$P\ M * (\Box \forall v\ \text{vs}. P(v :: \text{vs}) \multimap \text{WPE } b[x := v]\ \{\theta(r.\theta(r))\}) \\ * (P[] \multimap \Phi(\text{val } ())) \vdash \text{WPE for } x \text{ in } M \text{ do } b\ \{\Phi\},$$

where $\theta(\text{val } _) = P\ \text{vs}$ and $\theta(\text{exn } s\ v) = \Phi(\text{exn } s\ v)$; specialising $P\ M := \ulcorner \text{Forall } Q\ M \urcorner$ yields the per-element forall loop the generator emits. A dedicated heap-counter while rule closes counting loops by Löb induction. Each rule has a matching *stage tactic* (Section 3.7): pure_step fires LET/OP/IFT/IFF/TRYV/TRYC, heap_load/store/alloc the heap rules, call_opaque/call_transparent the call rules, raise_step RAISE/UNWIND, and loop_unfold the loop rules.

3.4 Contracts compile to iProp, composed with the separating conjunction

Contracts remain ordinary Python – plain assert statements (leading = precondition, trailing-before-return = postcondition, in-loop = invariant) or a verifier-only axiomander: docstring. A shared

Table 1: Contract fragments compiled to iProp.

Python contract fragment	iProp
assert $x \geq 1$	$\ulcorner x \geq 1 \urcorner$
assert implies(A, B)	$\ulcorner A \rightarrow B \urcorner$
assert all($x \neq i$ for i in range($\emptyset, 10$))	$\ulcorner \forall i, 0 \leq i < 10 \rightarrow x \neq i \urcorner$
owns(box)	$\text{box.value} \mapsto v$ (a resource)

ContractLinter parses them into a contract IR; a backend-specific pass (contract_ir_iris) compiles that IR to iProp. Pure facts are injected with the persistent embedding $\ulcorner P \urcorner$; resource facts compile to points-to assertions.

The decisive design choice is that **the default composition of contract clauses is the separating conjunction $*$, not $\wedge$** . When a precondition or postcondition is built from several asserts, they are joined with $*$. For the current pure-integer fragment this costs nothing, since $\ulcorner P \urcorner * \ulcorner Q \urcorner \vdash \ulcorner P \wedge Q \urcorner$; but it means the *same* composition rule already accounts for disjoint resources, and it is the connective that will let us scale to invariants and atomic updates without changing the front end.

A typical mixed pure/resource contract, written by the user with no Iris notation:

```
def bump(box: Box) -> int:
    """
    axiomander:
        requires:
            owns(box)
            box.value >= 0
        modifies:
            box.value
        ensures:
            owns(box)
            box.value == old(box.value) + 1
            result == box.value
    """
    box.value = box.value + 1
    return box.value
```

compiles to the Hoare triple

$$\{ \text{box.value} \mapsto n * \ulcorner n \geq 0 \urcorner \} \quad \text{bump}(\text{box}) \\ \{ r. \text{box.value} \mapsto n+1 * \ulcorner r = n+1 \urcorner \}.$$

owns(box) plus modifies: box.value is the resource footprint; the user never writes $*$, $\mapsto$, masks, or proof-mode tactics.

3.5 The frame rule is free, and so is composition

Because assertions are iProp, the structural frame rule

$$\frac{\{P\} c \{Q\}}{\{P * R\} c \{Q * R\}}$$

holds without a side condition: any resource R describing a disjoint region of the heap is carried across c by the separating conjunction, automatically. Compared with Section 2, the entire clobber/ $\forall v \notin \text{writes}$ apparatus, and the per-callee per-variable frame-lemma generator that papered over it, simply disappear. Locality stops being something we encode and becomes the shape of the logic. This is the single biggest reason for the migration: *composability is sound, not merely convenient*.

$$\begin{array}{c}
\textbf{Pure reduction } e \rightsquigarrow e' \\
\text{LET } \frac{}{\text{let } x=v \text{ in } e \rightsquigarrow e[x:=v]} \quad \text{OP } \frac{}{op(v_1, v_2) \rightsquigarrow \text{binop}(op, v_1, v_2)} \quad \text{IFT } \frac{}{\text{if true then } e_1 \text{ else } e_2 \rightsquigarrow e_1} \quad \text{IFF } \frac{}{\text{if false then } e_1 \text{ else } e_2 \rightsquigarrow e_2} \\
\text{TRYV } \frac{}{\text{try } v \text{ catch } x \Rightarrow h \rightsquigarrow v} \quad \text{TRYC } \frac{}{\text{try (raise } v) \text{ catch } x \Rightarrow h \rightsquigarrow h[x:=v]} \quad \text{UNWIND } \frac{\text{neutral}(K)}{K[\text{raise } v] \rightsquigarrow \text{raise } v} \\
\text{WHILE } \frac{}{\text{while } e_1 \rightsquigarrow \text{if } e_1 \text{ then } (e_2; \text{while } e_1 \rightsquigarrow e_2) \text{ else } ()} \quad \text{FORNIL } \frac{}{\text{for } x \text{ in } [] \text{ do } b \rightsquigarrow ()} \\
\text{FORCONS } \frac{}{\text{for } x \text{ in } (v::vs) \text{ do } b \rightsquigarrow b[x:=v]; \text{for } x \text{ in } vs \text{ do } b} \\
\textbf{Head reduction } (e, \sigma) \rightarrow (e', \sigma'; \vec{ef}) \quad \text{and} \quad \textbf{context decomposition} \\
\text{LOAD } \frac{\sigma(l) = v}{(!l, \sigma) \rightarrow (v, \sigma)} \quad \text{STORE } \frac{l \in \text{dom } \sigma}{(l := v, \sigma) \rightarrow ((), \sigma[l \mapsto v])} \quad \text{ALLOC } \frac{\sigma(l) = \perp}{(\text{ref } v, \sigma) \rightarrow (l, \sigma[l \mapsto v])} \\
\text{CALLSPEC } \frac{\text{entries}(g) = \text{FunSpec } P \ Q \quad P \vec{v} \quad Q \vec{v} v}{(g(\vec{v}), \sigma) \rightarrow (v, \sigma)} \quad \text{CALLDEF } \frac{\text{entries}(g) = \text{FunDef } \vec{x} \ b \quad |\vec{v}| = |\vec{x}|}{(g(\vec{v}), \sigma) \rightarrow (b[\vec{x}:=\vec{v}], \sigma)} \quad \text{CTX-P } \frac{e \rightsquigarrow e'}{(K[e], \sigma) \rightarrow (K[e'], \sigma)} \\
\text{CTX-H } \frac{(e, \sigma) \rightarrow (e', \sigma'; \vec{ef})}{(K[e], \sigma) \rightarrow (K[e'], \sigma'; \vec{ef})}
\end{array}$$

Figure 2: Snakelet operational semantics. Pure steps are heap-independent and deterministic; head steps touch the heap or the contract table entries. CALLSPEC fires only inside the callee’s precondition P (calling outside P is stuck), making the precondition a proof obligation on the caller. $\vec{ef}$ collects forked threads (empty except for fork in the core development).

3.6 Opaque vs. transparent calls: stubbing as a logical primitive

Calls are dispatched through a **contract table** (an Iris FunCtx) that maps each callee to its specification. SnakeletWp provides two call rules:

- **Transparent call** – the callee’s body is available and is unfolded; used for first-party functions being verified in the same run.
- **Opaque call** – only the callee’s *contract* is available; the caller consumes the callee’s precondition (giving up the relevant resources), receives its postcondition, and frames the rest. Schematically, for a callee with spec $\{P_g\} g \{Q_g\}$:

$$\frac{P_g * R \quad \forall r. (Q_g * R) \vdash \text{wp } K \{ \Phi \}}{\text{wp } (\text{call } g; K) \{ \Phi \}}$$

The opaque rule is exactly **stubbing as a logical primitive**. A library function is given a contract – in a .pyi stub or an axiomander: docstring – and that contract enters the FunCtx; callers prove against it via the opaque rule, and the body is never required. It is also exactly **modular verification under iteration**: a transparent callee can be re-declared opaque once its contract is stable, and its callers’ proofs depend only on the contract, so a body edit does not disturb them. The FunCtx entry’s hash is the summary that our incremental cache keys on.

3.7 Staged, syntax-directed proofs with independent failure

SnakeletWp ships a small instruction set of **stage tactics** – pure_step, case_bool, call_opaque, call_transparent, finish_pure, and a precondition-discharge stage call_opaque_pre – and the proof generator (iris_proof_gen) emits exactly one stage per IR node, chosen by the node’s syntax and the callee’s table entry. The generator performs no symbolic execution and needs no knowledge of intermediate values: each stage tactic extracts what it needs from the Iris goal at proof time. Loops are handled by an iLöb-based invariant stage, with the invariant taken verbatim from the user’s in-loop assert. The proof script is the trace.

The payoff for the LLM loop is that **stages fail independently with classifiable errors**. When call_opaque_pre cannot discharge a callee precondition (e.g. nonlinear arithmetic, a string fact), the residual is exported to SMT; an UNSAT result is imported as Axiom smt_ax_N : ... and the stage is regenerated as call_opaque_pre (exact smt_ax_N). A pure postcondition that finish_pure cannot close (reflexivity/lia) is likewise an SMT candidate, and beyond SMT the *structured residual goal* – a named stage plus the goal state from coq-lsp, rather than a stderr regex – is what the LLM oracle receives. Small named obligations give better SMT performance, better LLM prompts, per-stage caching, and parallelism, and they mean a failed attempt yields a reusable artifact rather than a dead end.

$\text{VAL} \frac{}{\Phi(\text{val } v) \vdash \text{WPE } v \llbracket \Phi \rrbracket}$	$\text{RAISE} \frac{}{\Phi(\text{exn } s \ v) \vdash \text{WPE raise } (\text{exn } s \ v) \llbracket \Phi \rrbracket}$	$\text{WAND} \frac{}{\text{WPE } e \llbracket \Phi \rrbracket * (\forall r. \Phi r \multimap \Psi r) \vdash \text{WPE } e \llbracket \Psi \rrbracket}$
$\text{LET} \frac{}{\triangleright \text{WPE } e[x:=v] \llbracket \Phi \rrbracket \vdash \text{WPE let } x=v \text{ in } e \llbracket \Phi \rrbracket}$	$\text{LETRAISE} \frac{}{\Phi(\text{exn } s \ v) \vdash \text{WPE let } x=\text{raise } (\text{exn } s \ v) \text{ in } e \llbracket \Phi \rrbracket}$	
$\text{OP} \frac{}{\triangleright \text{WPE binop}(op, v_1, v_2) \llbracket \Phi \rrbracket \vdash \text{WPE } op(v_1, v_2) \llbracket \Phi \rrbracket}$	$\text{IFT} \frac{}{\triangleright \text{WPE } e_1 \llbracket \Phi \rrbracket \vdash \text{WPE if true then } e_1 \text{ else } e_2 \llbracket \Phi \rrbracket}$	
$\text{IF} \frac{}{\triangleright \text{WPE } e_2 \llbracket \Phi \rrbracket \vdash \text{WPE if false then } e_1 \text{ else } e_2 \llbracket \Phi \rrbracket}$	$\text{TRYV} \frac{}{\triangleright \text{WPE } v \llbracket \Phi \rrbracket \vdash \text{WPE try } v \text{ catch } x \Rightarrow h \llbracket \Phi \rrbracket}$	
$\text{TRYC} \frac{}{\triangleright \text{WPE } h[x:=v] \llbracket \Phi \rrbracket \vdash \text{WPE try } (\text{raise } v) \text{ catch } x \Rightarrow h \llbracket \Phi \rrbracket}$	$\text{BIND} \frac{\text{neutral}(K)}{\text{WPE } e \llbracket \text{bindp } K \Phi \rrbracket \vdash \text{WPE } K[e] \llbracket \Phi \rrbracket}$	
$\text{LOAD} \frac{}{l \mapsto v * \triangleright (l \mapsto v \multimap \Phi(\text{val } v)) \vdash \text{WPE } !l \llbracket \Phi \rrbracket}$	$\text{STORE} \frac{}{l \mapsto w * \triangleright (l \mapsto v \multimap \Phi(\text{val } ())) \vdash \text{WPE } l := v \llbracket \Phi \rrbracket}$	
$\text{ALLOC} \frac{}{\triangleright (\forall l. l \mapsto v \multimap \Phi(\text{val } l)) \vdash \text{WPE ref } v \llbracket \Phi \rrbracket}$	$\text{FORNIL} \frac{}{\triangleright \Phi(\text{val } ()) \vdash \text{WPE for } x \text{ in } [] \text{ do } b \llbracket \Phi \rrbracket}$	
$\text{CALLOPAQUE} \frac{\text{entries}(g) = \text{FunSpec } P \ Q \quad P \vec{v}}{\triangleright (\forall v. \ulcorner Q \vec{v} v^{-1} \urcorner \multimap \Phi(\text{val } v)) \vdash \text{WPE } g(\vec{v}) \llbracket \Phi \rrbracket}$	$\text{CALLDEF} \frac{\text{entries}(g) = \text{FunDef } \vec{x} \ b \quad \vec{v} = \vec{x} }{\triangleright \text{WPE } b[\vec{x}:=\vec{v}] \llbracket \Phi \rrbracket \vdash \text{WPE } g(\vec{v}) \llbracket \Phi \rrbracket}$	
$\text{WHILE} \frac{}{\triangleright \text{WPE if } e_1 \text{ then } (e_2 ; \text{while } e_1 \ e_2) \text{ else } () \llbracket \Phi \rrbracket \vdash \text{WPE while } e_1 \ e_2 \llbracket \Phi \rrbracket}$		
$\text{FORCONS} \frac{}{\triangleright \text{WPE } b[x:=v] ; \text{for } x \text{ in } vs \text{ do } b \llbracket \Phi \rrbracket \vdash \text{WPE for } x \text{ in } (v :: vs) \text{ do } b \llbracket \Phi \rrbracket}$		

Figure 3: SnakeletWp weakest-precondition rules. The postcondition $\Phi : \text{Result} \rightarrow i\text{Prop}$ ranges over $r ::= \text{val } v \mid \text{exn } s \ v$; $\triangleright$ is the Iris later modality, $*$ the separating conjunction, $\multimap$ the magic wand, and $\ulcorner \cdot \urcorner$ the persistent embedding of a pure proposition. **BIND uses the Result-transformer `bindp` (text); the wider **WHILE**/**FORCONS** rules are shown beneath the grid. Every rule is machine-checked.**

3.8 What the Iris choice does and does not change

The migration is deliberately confined to the proof backend. The `ContractLinter`, the SMT export (`smt_export`, `theory_smt`), the LLM oracle, and the function-level incremental cache are all backend-agnostic; the contract IR diverges only at the final rendering step (flat-store to `coq` vs. `contract_ir_iris.iris_prop`). Iris *adds* per-stage hashing for finer-grained reuse but does not replace the cache. From the user’s side, nothing changes: contracts are still decorator-free Python, and `owns(...)` is the only new, entirely optional, resource vocabulary.

4 Discussion and relation to prior work

Axiomander’s separation-logic interpretation of object fields and its opaque call rule are textbook Iris; the novelty is not in the logic but in the *system context*. To our knowledge the combination is unusual: (i) the assertion logic is never exposed to the user – contracts are plain Python and the separating conjunction is the implicit default composition; (ii) the proof obligations are generated and largely discharged by an LLM operating on structured residual goals, with Iris/Rocq as the trust kernel; and (iii) the same `FunCtx` opaque rule serves simultaneously as modular verification, incremental-reuse

boundary, and library stubbing mechanism. We see Axiomander as a stress test of the claim that Iris’s framing and ownership story is the right substrate for *automated*, composition-heavy verification, not only for expert-driven interactive proof.

Our experience also offers a small, concrete data point: building a verifier without a substructural logic and then retrofitting locality (Section 2) is costly and bounded; starting from Iris (Section 3) makes the very properties our LLM loop depends on – frame, stub, reuse – definitional.

5 Status and open questions for the community

Status (June 2026). Snakelet and SnakeletWp are complete and free of `Admitted` across the rule set of Figures 2–3: the core fragment (binop, let, if, load/store/alloc, opaque/transparent call) and the exception-aware extension with its `Result`-typed postcondition and loop rules, whose eight-lemma convergence gate is `Qed`. The staged generator, the end-to-end Python-to-Iris pipeline (sharing the `ContractLinter`), the contract-to-*iProp* compiler, and the SMT-axiom escalation slot are in place, with tens of Iris-specific tests passing alongside the mature flat-store backend. Remaining work wires the exception calculus into the Python lowering and

extends the fragment to loop invariants over shared disjoint memory, in-place data-structure mutation, and type-aware reasoning over strings/floats/isinstance/None.

We would value the community's view on:

- (1) **Ownership inference from modifies.** We currently derive a footprint from explicit `owns(...)` plus `modifies:..`. How far can footprints be *inferred* for an LLM-generated body without burdening the user, and what is the right failure mode when inference is ambiguous?
- (2) **Specs as the LLM's search space.** The LLM proposes both code and helper contracts. Are there idioms (magic wands, representation predicates, abstract predicates with `FunCtx`-level abstraction) that are especially amenable to automatic *generation*, as opposed to automatic *use*?
- (3) **Residual goals as oracle inputs.** We feed a failed stage's goal state to an LLM. Which Iris proof-mode presentations of a residual (hypothesis naming, spatial/persistent split, masks elided) make the best prompts?
- (4) **Concurrency on the same front end.** Our composition default is already `*`. What is the minimal extension of the

decorator-free contract surface that could express invariants/atomic updates without exposing Iris syntax to the user?

- (5) **Trust surface.** With SMT-discharged facts imported as axioms and LLM-generated proofs re-checked by Rocq, the kernel remains the authority but the axiom slots are a trust boundary. What discipline (e.g. logged query hashes, replay) would the community consider acceptable for reproducibility?

Artifact pointers

This paper summarises material developed in the Axiomander repository: the Iris migration plan and current state; the Iris backend prototype (the bump/owns example and resource classification); the per-callee frame lemmas in the flat-store backend (Section 2); the staged proof engineering notes; the incremental-verification cache design; and the companion whitepaper and slides motivating vericoding.